

(ملاحظة الصور و ملفات الاختبار موجودة في مجلد يحتوي مجلد لكل طلب فيه صور او ملفات الخرج)

الطلب الأول : حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

لإثبات وجود مشكلة التزامن وتأثيرها المباشر على سلامة البيانات، قمنا بإنشاء مسار تجريبي غير آمن (Unsafe Experimental Path) مؤقتًا داخل المتحكم `TestCheckoutController.php` لمحاكاة السلوك الافتراضي للنظام قبل معالجته، وسيتم إزالة هذا المسار بعد التوثيق. وايضا حقل المخزون (stock) في قاعدة البيانات معرّف بنوع unsigned، مما يمنع قيمته ان تكون سالبة فقط يصبح صفر؛ ومع ذلك، فإن المشكلة الأساسية لم تكن في القيمة السالبة، بل في توليد أوامر شراء ناجحة (Orders) تتجاوز العدد الفعلي المتاح في المخزون، مثل إنشاء عدة طلبات لمنتج لا تتوفر منه سوى قطعة واحدة.

ولحل هذه المشكلة ، تم اعتماد أسلوب العمليات الذرية المشروطة (Conditional Atomic Decrement) المدعومة على مستوى قاعدة البيانات ومغلقة داخل عملية مركبة (Database Transaction) يضمن هذا الحل دمج عمليتي التحقق من الشرط والخصم الفعلي في استعلام (Query) واحد غير قابل للانقسام لمنع التداخل بين الطلبات المتزامنة، يتم التنفيذ برمجياً كالتالي:

```
DB::transaction(function () use ($productId, $qty) {  
    // الخصم الذري المشروط يمنع أي تداخل بين الطلبات المتزامنة  
    $updated = Product::where('id', $productId)  
        ->where('stock', '>=', $qty)  
        ->decrement('stock', $qty);  
  
    if (! $updated) {  
        throw new Exception('Out of stock or insufficient  
inventory.');    }  
});
```

لماذا اخترنا هذا الحل:

لأنه يضمن عدم حدوث تعارض، وكفاءته عالية؛ وهو أفضل من القفل المتشائم إذا كانت الحالة بسيطة بدون كوبونات أو ما شابه، حيث يكتفي بقفل السطر ويضمن أداءً عالياً.

نتائج الاختبار:

تم استخدام سكربت محاكاة التزامن المكثف `concurrent_checkout_test.php` عبر أداة JMeter

بـ 5 Threads متزامنة في نفس الأجزاء من الثانية، وأظهرت النتائج المخبرية المصورة ما يلي:

- **في المسار غير المحمي (قبل الحل):** عند اختبار 5 حالات شراء متزامنة لمنتج ذي مخزون محدود (قطعة واحدة)؛ كانت النتيجة نجاح الحالات الخمس بالكامل وتوليد 5 أوامر شراء (Overselling) على الرغم من عدم كفاية المخزون المادي.
- **في المسار المحمي (بعد الحل):** عند إعادة تشغيل نفس السيناريو، فشلت 4 عمليات وتم رفضها بأمان، ولم تنجح سوى عملية واحدة فقط (1) وهي التي التقطت قفل السطر أولاً واستحقت المخزون المتاح.

الطلب الثاني: إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

كيف تم الحل:

تم تنفيذ آلية Rate Limiting ديناميكية داخل `RouteServiceProvider.php`. بدلاً من الاعتماد على قيم ثابتة، يتم احتساب الحدود برمجياً استناداً إلى إعدادات السيرفر الممررة عبر ملف `env`. مثل عدد الـ Threads في Apache المستضاف محلياً على XAMPP، ومتوسط زمن الاستجابة، ونسبة الاستهلاك المستهدفة. تم توزيع هذه القدرة الاستيعابية على مجموعات حماية مخصصة: `auth`، `cart`، `checkout`، و `products`.

لماذا اخترنا هذا الحل:

لأنه يربط قيود الطلبات بقدرة السيرفر الفعلية، ويسهل تعديل الإعدادات من ملف الانفايرومنت دون تغيير الكود، وهو يعطيني مرونة عند تغيير حجم السيرفر.

كيف تمت عملية التحقق:

(ملاحظة: في البداية كنت قد وضعت الريت لمتنغ ستاتك ولكن عدلته عند حل الطلب الرابع).

تمت مقارنة سلوك النظام تحت ضغط اختبار حمل مكثف (Load Test) يصل إلى 230 مستخدم وهمي متزامن (Max VUs) عبر أداة k6:

- **قبل تطبيق الحل (بدون قيود):** تكدست الطلبات ووصل متوسط زمن الاستجابة إلى 19.17 ثانية، وبلغت ذروة التأخير في الطابور p(95) حوالي 31.87 ثانية. تسبب هذا الضغط غير المحدود في استهلاك مفرط للموارد؛ حيث التهم النظام ذاكرة عشوائية قاسية بلغت ذروتها (Max RAM) حوالي MB 2480.61 مع متوسط استهلاك معالج بـ 16.68% (وذروة وصلت إلى 31.77%).
- **بعد تطبيق الحل الديناميكي:** عند تشغيل نفس الاختبار، نجح النظام في صد الضغط وحماية الموارد عن طريق تفعيل الـ Too Many Requests 429؛ حيث رصد الاختبار خروج 44 طلب مقيد للـ auth، و 34 للـ cart، و 28 للـ checkout.
- **النتيجة على الموارد:** انخفض متوسط استهلاك المعالج (Avg CPU) بشكل ملحوظ ليصل إلى 5.71% (بذروة 25.34%)، وهبط استهلاك الذاكرة العشوائية الأقصى إلى MB 1881.07، مما يثبت نجاح الـ Throttling في حماية الخادم من الانهيار وتوفير عزل كامل للموارد.

الطلب الثالث: المعالجة غير المتزامنة (Asynchronous Queues)

كيف تم الحل:

تم الانتقال بالكامل من المحرك المتزامن (sync) إلى محرك الطوابير المعتمد على قاعدة البيانات (database driver) لتعمل المهام الثقيلة في الخلفية. تم تحويل العمليات المستهلكة للوقت إلى ثلاث وظائف خلفية مستقلة (Background Jobs):

- **ProcessOrderJob:** لمعالجة الدفع وتحديثات حالات الطلب بعد الـ checkout.
 - **GenerateOrderSummaryJob:** لتوليد ملفات ملخصات الطلبات للمشتريين.
 - **GenerateDailySalesReportJob:** لتجميع التقارير البيعية اليومية الضخمة.
- تم إضافة جدول jobs عبر migration مخصص، وضبط منطق تراجع المحاولات المتدرج تلقائياً عند الفشل (retry/backoff).

لماذا اخترنا هذا الحل:

- لا يعتمد على تشغيل خدمات خارجية إضافية مما يجعله مناسباً لحجم المشروع الحالي.
- يسمح بمراقبة ومتابعة المهام المخزنة برؤية مباشرة داخل جداول قاعدة البيانات.

كيف تم التحقق (الأدلة الرقمية من اختبارات الأداء):

تم الفحص بواسطة سيناريو تحميل معتدل متوازن (env LOAD_PROFILE=moderate-- عبر أداة k6 للتأكد من انتقال المعالجة إلى الخلفية ومراقبة الموارد عبر [project_resource_monitor.php](#) الممتد على مدار 90 ثانية:

- **تحسن أوقات الاستجابة:** انخفض زمن الاستجابة العام لـ p(95) بشكل حاد ليصل إلى 12.12 ثانية فقط (حيث سجلت مسارات المنتجات والسلة حوالي 10.79 ثانية)، ولم يعد العميل ينتظر انتهاء المعالجة على خادم الويب مباشرة.
- **كفاءة استهلاك الموارد:** أظهر مراقب الموارد ثباتاً ممتازاً؛ حيث استقر متوسط استهلاك المعالج (Avg CPU) عند 3.26% فقط (بذروة ترحيل مؤقتة بلغت 52.58%)، وحافظت الذاكرة على استقرار ممتاز بمتوسط استهلاك 1423.31 MB

الطلب الرابع: معالجة البيانات الضخمة على دفعات (Batch Processing)

كيف تم الحل:

تم تنفيذ المعالجة الدفعية داخل [GenerateDailySalesReportJob.php](#) باستخدام خاصية تفتيت البيانات chunk(100) حتى تُعالج سجلات الطلبات على دفعات صغيرة بدلاً من تحميل كل السجلات الضخمة دفعة واحدة في الذاكرة. ولإظهار الأثر عملياً تمت إضافة:

- كلاس [ResourceMonitor.php](#) لمراقبة الأداء محلياً.
- سكربت [batch_processing_demo.php](#) لتوليد بيانات ضخمة وتشغيل التقرير.
- سجلات (Logs) واضحة تظهر تقسيم العمليات بصيغة Chunk #1, Chunk #2.

لماذا اخترنا هذا الحل:

يضمن تقليل استهلاك الذاكرة العشوائية للحد الأدنى، ويمنع حدوث خطأ نفاذ الذاكرة الشهير (Fatal Error: Allowed Memory Size Exhausted) عند نمو البيانات، ويحقق الهدف الأكاديمي بوضوح في سجلات النظام.

كيف تم التحقق:

أظهرت سجلات النظام (laravel.log) تقسيم البيانات بنجاح وثبات كامل للموارد:

Processing Sales Report Batch - Chunk #1 | Orders in this batch: 100

Processing Sales Report Batch - Chunk #2 | Orders in this batch: 100

...

الطلب الخامس: توزيع الأحمال (Load Distribution)

كيف تم الحل:

لتوضيح كيفية تشغيل التطبيق على أكثر من خادم وكيفية توزيع الأحمال بينها، قمنا بتجهيز بيئة اختبار تحاكي خوادم الإنتاج وفق المعايير التالية:

- **البنية:** استخدمنا موازن الأحمال في أباتشي mod_proxy_balancer ك Load Balancer أمامي يقود إلى مجموعتين من خوادم التطبيق الخلفية تعمل على منافذ مختلفة.
- **إثبات الاتصال:** أضفنا نهاية مسار اختبارية /server-info تعيد هوية الخادم الحالي الذي استقبل الطلب (PID).
- **الاستراتيجية المتبعة:** تم تطبيق سياسة التوزيع الدائري Round-Robin مع فحوصات الصحة (Health Checks) لتبسيط العرض الأكاديمي.

كيف تم التحقق:

عند تشغيل سكربت `k6_load_balance_demo.js` ليصوب طلبات متتالية إلى نقطة التحميل الأمامية، أظهرت النتائج توزيعاً متقارباً ومتوازناً للطلبات بين الخوادم الخلفية بنسب (52% مقابل 48%)، مما يبرهن هندسياً أن التوزيع يقلل الضغط عن الخادم الواحد ويزيد من توفر النظام (High Availability).